Computer Programming 1

Functions (part 2)

Outline

- (Quick) Recap *(part 1)*
  - Introduction (Functions, Library of functions, Procedures, …)
  - Definition, Declaration and Call of Functions
  - Function parameters and local variables
- Parameters passing *(part 2)*
- Recursive functions *(part 3)*

Outline

- **(Quick) Recap *(part 1)***
  - Introduction (Functions, Library of functions, Procedures, …)
  - Definition, Declaration and Call of Functions
  - Function parameters and local variables
- Parameters passing *(part 2)*
- Recursive functions *(part 3)*

Introduction to the functions and Libraries

Function

- In a program, the use of functions is convenient to **structure the code** by grouping its parts in autonomous sub-programs, which are executable where it is required.
- A function can be used repeatedly in a program, or in different programs

Library

- It is a set of **precompiled functions**.
- A library consists of a pair of files:
  - a header file containing the declarations of the sub-programs themselves
  - an object file containing the compiled functions
- The **#include** directive introduces in the program calls to the functions of the library
  - E.g., **#include** <libraryname>
  - E.g., **#include** “libraryname.h”

Functions: declarations, definition and call

- Definition
  - Syntax: type id(type1 id1, … typeN idN) { … }
  - Id1, …, idN are the formal parameters of the function
  - E.g., double pow(double x, double y) { … }
- Declaration
  - Syntax: type id(type1 [id1], … typeN [idN]);
  - id1, …, idN are optional parameters
  - E.g., double pow(double, double e);
  - It is used to "recall" a definition made elsewhere and allow its use!
- Call
  - Syntax: id (exp1, …, expN)
  - exp1, … , expN are the actual/current parameters of the call
  - E.g., x = pow(2.0*y, 3.0)

Example

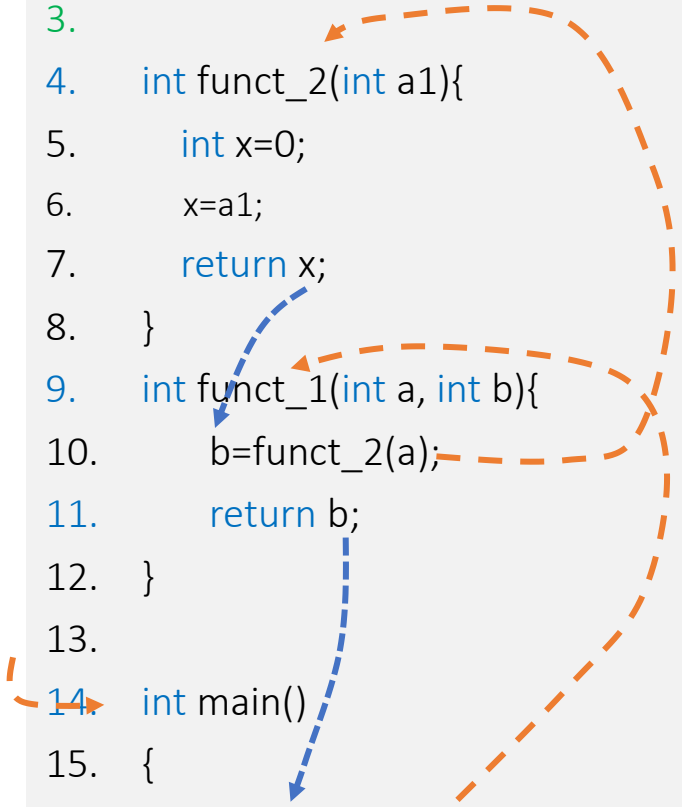
```
// definition
        int gcd(int a, int b){
          // body of the function…..
          …
          return 0;
        }
// declaration
        int gcd(int, int);
// call
        int ret = gcd(24,36);
```

Note

- The actual parameters exp1, ..., expN of the call must be compatible for number, order and respectively by type to the corresponding formal parameters!
- The ***signature of a function*** is composed of: (i) the function name; (ii) number, order and type of formal parameters
  - Note that the type of the return parameter is not part of the signature

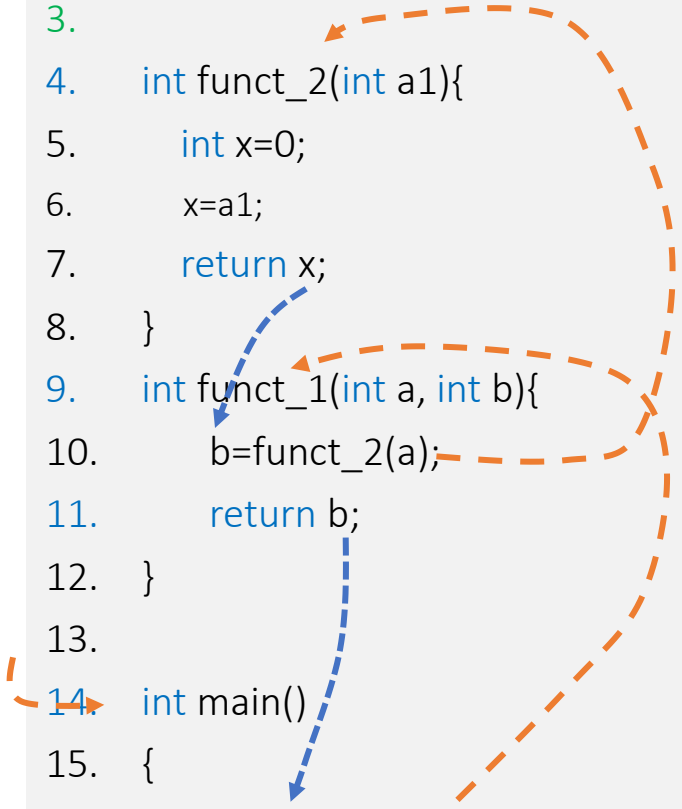
Examples of code structure

```
1.  using namespace std;
2.  #include <iostream>
3.
4.  int funct_2(int a1){
5.      int x=0;
6.      x=a1;
7.      return x;
8.  }
9.  int funct_1(int a, int b){
10.     b=funct_2(a);
11.     return b;
12. }
13.
14. int main()
15. {
16.     int res=funct_1(24,36);
17.     cout << res << endl;
18. }
```



Examples of code structure

```
1.  using namespace std;
2.  #include <iostream>
3.
4.  int funct_2(int a1){
5.      int x=0;
6.      x=a1;
7.      return x;
8.  }
9.  int funct_1(int a, int b){
10.     b=funct_2(a);
11.     return b;
12. }
13.
14. int main()
15. {
16.     int res=funct_1(24,36);
17.     cout << res << endl;   //24
18. }
```



Examples of code structure

```
1.  using namespace std;
2.  #include <iostream>
3.
4.  int funct_2(int a1){
5.      int x=0;
6.      x=a1;
7.      return x;
8.  }
9.  int funct_1(int a, int b){
10.     b=funct_2(a);
11.     return b;
12. }
13.
14. int main()
15. {
16.     int res=funct_1(24,36);
17.     cout << res << endl;   //24
18. }
```

```
1.  using namespace std;
2.  #include <iostream>
3.
4.  int funct_1(int a, int b){
5.      b=funct_2(a);
6.      return b;
7.  }
8.  int funct_2(int a1){
9.      int x=0;
10.     x=a1;
11.     return x;
12. }
13.
14. int main()
15. {
16.     int res=funct_1(24,36);
17.     cout << res << endl;   //24
18. }
```

Attention: funct_2 is called before the definition

Examples of code structure

```
1.  using namespace std;
2.  #include <iostream>
3.
4.  int funct_2(int a1){
5.      int x=0;
6.      x=a1;
7.      return x;
8.  }
9.  int funct_1(int a, int b){
10.     b=funct_2(a);
11.     return b;
12. }
13.
14. int main()
15. {
16.     int res=funct_1(24,36);
17.     cout << res << endl;   //24
18. }
```

```
1.  using namespace std;
2.  #include <iostream>
3.
4.  int funct_1(int, int);
5.  int funct_2(int);
6.  int funct_1(int a, int b){
7.      b=funct_2(a);
8.      return b;
9.  }
10. int funct_2(int a1){
11.     int x=0;
12.     x=a1;
13.     return x;
14. }
15.
16. int main()
17. {
18.     int res=funct_1(24,36);
19.     cout << res << endl;   //24
20. }
```

It works!

Examples of code structure

```
1.  using namespace std;
2.  #include <iostream>
3.
4.  int funct_2(int a1){
5.      int x=0;
6.      x=a1;
7.      return x;
8.  }
9.  int funct_1(int a, int b){
10.     b=funct_2(a);
11.     return b;
12. }
13.
14. int main()
15. {
16.     int res=funct_1(24,36);
17.     cout << res << endl;   //24
18. }
```

```
1.  using namespace std;
2.  #include <iostream>
3.
4.  int funct_2(int);
5.  int funct_1(int, int);

6.  int funct_1(int a, int b){
7.      b=funct_2(a);
8.      return b;
9.  }
10. int funct_2(int a1){
11.     int x=0;
12.     x=a1;
13.     return x;
14. }
15.
16. int main()
17. {
18.     int res=funct_1(24,36);
19.     cout << res << endl;   //24
20. }
```

It works!

Step-by-step execution of a function

Execution steps:

1) the main function is executed
2) two local variables are defined: p1 and p2
3) the function gcd is called with p1 and p2 as current parameters
   - the current parameter values p1 and p2 are calculated (the order is not guarantee)
   - the values of current parameters are copied, in order, in the formal parameters a and b (*call by value*)
4) the function is executed and the values of a, b, and of the local variable res are modified (p1 and p2 remain with their original value)
5) the gcd function returns to the calling program the value of the expression that appears in the return statement (res)
6) The result of the function execution is copied in the variable res1
7) The value of the variable res1 is printed

```
using namespace std;
#include <iostream>

int gcd(int a, int b){
    int res=0;
    … //gcd computation a%b …
     return res;
}

int main()
{
     int p1=12*2;
     int p2=36
     int res1=gcd(p1,p2);
     cout << res1 << endl;
}
```

Step-by-step execution of a function

Execution steps:

1) the main function is executed
2) two local variables are defined: p1 and p2
3) the function gcd is called with p1 and p2 as current parameters
   - the current parameter values p1 and p2 are calculated (the order is not guarantee)
   - the values of current parameters are copied, in order, in the formal parameters a and b (*call by value*)
4) the function is executed and the values of a, b, and of the local variable res are modified (p1 and p2 remain with their original value)
5) the gcd function returns to the calling program the value of the expression that appears in the return statement (res)
6) The result of the function execution is copied in the variable res1
7) The value of the variable res1 is printed



```
using namespace std;
#include <iostream>

int gcd(int a, int b){
    int res=0;
    … //gcd computation a%b …
     return res;
}

int main()
{
     int p1=12*2;
     int p2=36;
     int res1=gcd(p1,p2);
     cout << res1 << endl;
}
```

Step-by-step execution of a function

Execution steps:

1) the main function is executed
2) two local variables are defined: p1 and p2
3) the function gcd is called with p1 and p2 as current parameters
   - the current parameter values p1 and p2 are calculated (the order is not guarantee)
   - the values of current parameters are copied, in order, in the formal parameters a and b (*call by value*)
4) the function is executed and the values of a, b, and of the local variable res are modified (p1 and p2 remain with their original value)
5) the gcd function returns to the calling program the value of the expression that appears in the return statement (res)
6) The result of the function execution is copied in the variable res1
7) The value of the variable res1 is printed



```
using namespace std;
#include <iostream>

int gcd(int a, int b){
    int res=0;
    … //gcd computation a%b …
     return res;
}

int main()
{
     int p1=12*2;
     int p2=36;
     int res1=gcd(p1,p2);
     cout << res1 << endl;
}
```

Step-by-step execution of a function

Execution steps:

1) the main function is executed
2) two local variables are defined: p1 and p2
3) the function gcd is called with p1 and p2 as current parameters
   - the current parameter values p1 and p2 are calculated (the order is not guarantee)
   - the values of current parameters are copied, in order, in the formal parameters a and b (*call by value*)
4) the function is executed and the values of a, b, and of the local variable res are modified (p1 and p2 remain with their original value)
5) the gcd function returns to the calling program the value of the expression that appears in the return statement (res)
6) The result of the function execution is copied in the variable res1
7) The value of the variable res1 is printed

```
using namespace std;
#include <iostream>

int gcd(int a, int b){
    int res=0;
    … //gcd computation a%b …
     return res;
}

int main()
{
     int p1=12*2;
     int p2=36;
     int res1=gcd(p1,p2);
     cout << res1 << endl;
}
```

Step-by-step execution of a function

Execution steps:

1) the main function is executed
2) two local variables are defined: p1 and p2
3) the function gcd is called with p1 and p2 as current parameters
   - the current parameter values p1 and p2 are calculated (the order is not guarantee)
   - the values of current parameters are copied, in order, in the formal parameters a and b (*call by value*)
4) the function is executed and the values of a, b, and of the local variable res are modified (p1 and p2 remain with their original value)
5) the gcd function returns to the calling program the value of the expression that appears in the return statement (res)
6) The result of the function execution is copied in the variable res1
7) The value of the variable res1 is printed



```
using namespace std;
#include <iostream>

int gcd(int a, int b){
    int res=0;
    … //gcd computation a%b …
    return res;
}

int main()
{
    int p1=12*2;
    int p2=36;
    int res1=gcd(p1,p2);
    cout << res1 << endl;
}
```

Step-by-step execution of a function

Execution steps:

1) the main function is executed
2) two local variables are defined: p1 and p2
3) the function gcd is called with p1 and p2 as current parameters
   - the current parameter values p1 and p2 are calculated (the order is not guarantee)
   - the values of current parameters are copied, in order, in the formal parameters a and b (*call by value*)
4) the function is executed and the values of a, b, and of the local variable res are modified (p1 and p2 remain with their original value)
5) the gcd function returns to the calling program the value of the expression that appears in the return statement (res)
6) The result of the function execution is copied in the variable res1
7) The value of the variable res1 is printed



```
using namespace std;
#include <iostream>

int gcd(int a, int b){
    int res=0;
    … //gcd computation a%b …
    return res;
}

int main()
{
    int p1=12*2;
    int p2=36;
    int res1=gcd(p1,p2);
    cout << res1 << endl;
}
```

Step-by-step execution of a function

Execution steps:

1) the main function is executed
2) two local variables are defined: p1 and p2
3) the function gcd is called with p1 and p2 as current parameters
   - the current parameter values p1 and p2 are calculated (the order is not guarantee)
   - the values of current parameters are copied, in order, in the formal parameters a and b (*call by value*)
4) the function is executed and the values of a, b, and of the local variable res are modified (p1 and p2 remain with their original value)
5) the gcd function returns to the calling program the value of the expression that appears in the return statement (res)
6) The result of the function execution is copied in the variable res1
7) The value of the variable res1 is printed



```
using namespace std;
#include <iostream>

int gcd(int a, int b){
    int res=0;
    … //gcd computation a%b …
    return res;
}

int main()
{
    int p1=12*2;
    int p2=36;
    int res1=gcd(p1,p2);
    cout << res1 << endl;
}
```

Step-by-step execution of a function

Execution steps:

1) the main function is executed
2) two local variables are defined: p1 and p2
3) the function gcd is called with p1 and p2 as current parameters
   - the current parameter values p1 and p2 are calculated (the order is not guarantee)
   - the values of current parameters are copied, in order, in the formal parameters a and b (*call by value*)
4) the function is executed and the values of a, b, and of the local variable res are modified (p1 and p2 remain with their original value)
5) the gcd function returns to the calling program the value of the expression that appears in the return statement (res)
6) The result of the function execution is copied in the variable res1
7) The value of the variable res1 is printed



```
using namespace std;
#include <iostream>

int gcd(int a, int b){
    int res=0;
    … //gcd computation a%b …
    return res;
}

int main()
{
    int p1=12*2;
    int p2=36;
    int res1=gcd(p1,p2);
    cout << res1 << endl;
}
```

Step-by-step execution of a function

Execution steps:

1) the main function is executed
2) two local variables are defined: p1 and p2
3) the function gcd is called with p1 and p2 as current parameters
   - the current parameter values p1 and p2 are calculated (the order is not guarantee)
   - the values of current parameters are copied, in order, in the formal parameters a and b (*call by value*)
4) the function is executed and the values of a, b, and of the local variable res are modified (p1 and p2 remain with their original value)
5) the gcd function returns to the calling program the value of the expression that appears in the return statement (res)
6) The result of the function execution is copied in the variable res1
7) The value of the variable res1 is printed

```
using namespace std;
#include <iostream>

int gcd(int a, int b){
    int res=0;
    … //gcd computation a%b …
     return res;
}

int main()
{
     int p1=12*2;
     int p2=36;
     int res1=gcd(p1,p2);
     cout << res1 << endl;
}
```

Outline

- (Quick) Recap *(part 1)*
  - Introduction (Functions, Library of functions, Procedures, …)
  - Definition, Declaration and Call of Functions
  - Function parameters and local variables
- **Parameters passing *(part 2)***
- Recursive functions *(part 3)*

Parameter’ passing

Three methods to pass parameters to a function:

- by value
- by reference
- by pointer/address

Note: often the last two are confused in the literature, because they have similar purposes

Parameter passing by value (1)

- The definition of formal parameters is analogous to definition of variables
  - Syntax: type funct_name(type identifier, ...)
  - E.g., int fact (int n) {...}
  - Example of function call: fact(x);
- The formal parameters take the value of the current parameters
  - the current parameter can be an expression without an address
  - it can be of different compatible type → implicit conversion
- The information is (temporarily) duplicated

  → possible waste of CPU time and memory
- If I change the formal parameter, the actual/current parameter is not changed

  → passing of information only from the caller to the called

Examples:

{ ../FUNCTIONS/…}

Parameter passing by value (2)

- The definition of formal parameters is analogous to definition of variables
  - Syntax: type funct_name(type identifier, ...)
  - E.g., int fact (int n) {...}
  - Example of function call: fact(x);
- The formal parameters take the value of the current parameters
  - the current parameter can be an expression without an address
  - it can be of different compatible type → implicit conversion
- The information is (temporarily) duplicated

  → possible waste of CPU time and memory
- If I change the formal parameter, the actual/current parameter is not changed

  → passing of information only from the caller to the called

Examples:

{ ../FUNCTIONS/...}

Example

```
//function
int fun(int a)
{  …
   a = a+1
   … return a; }

//caller
int main(){
  …
  x=0;
  cout << x << endl;
  cout <<  fun(x) << endl;
  cout << x << endl;
  int y=fun(x)
  cout << y << endl;
  …
}
```

a=x=0

x=0

0

Parameter passing by value (2)

- The definition of formal parameters is analogous to definition of variables
  - Syntax: type funct_name(type identifier, ...)
  - E.g., int fact (int n) {...}
  - Example of function call: fact(x);
- The formal parameters take the value of the current parameters
  - the current parameter can be an expression without an address
  - it can be of different compatible type → implicit conversion
- The information is (temporarily) duplicated

  → possible waste of CPU time and memory
- If I change the formal parameter, the actual/current parameter is not changed

  → passing of information only from the caller to the called

Examples:

{ ../FUNCTIONS/...}

Example

```
//function
int fun(int a)
{ …
   a = a+1
   … return a; }

//caller
int main(){
  …
  x=0;
  cout << x << endl;
  cout <<  fun(x) << endl;
  cout << x << endl;
  int y=fun(x)
  cout << y << endl;
  …
}
```

a=x=0

a=1

1

x=0

0

1

0

Parameter passing by value (2)

- The definition of formal parameters is analogous to definition of variables
  - Syntax: type funct_name(type identifier, ...)
  - E.g., int fact (int n) {...}
  - Example of function call: fact(x);
- The formal parameters take the value of the current parameters
  - the current parameter can be an expression without an address
  - it can be of different compatible type → implicit conversion
- The information is (temporarily) duplicated

  → possible waste of CPU time and memory
- If I change the formal parameter, the actual/current parameter is not changed

  → passing of information only from the caller to the called

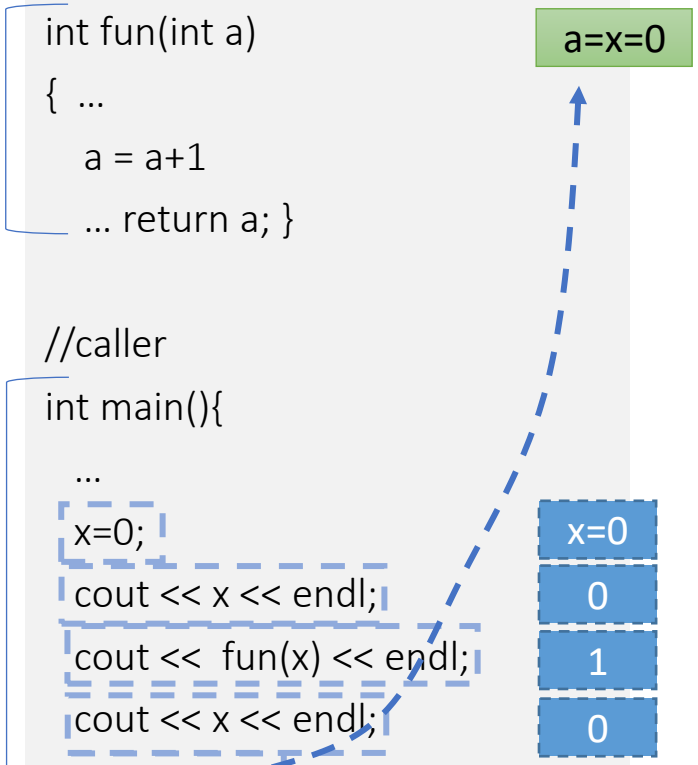
Examples:

{ ../FUNCTIONS/...}

Example

```
//function
int fun(int a)
{  …
   a = a+1
   … return a; }

//caller
int main(){
  …
  x=0;
  cout << x << endl;
  cout <<  fun(x) << endl;
  cout << x << endl;
  int y=fun(x)
  cout << y << endl;
  …
}
```



Parameter passing by value (2)

- The definition of formal parameters is analogous to definition of variables
  - Syntax: type funct_name(type identifier, ...)
  - E.g., int fact (int n) {...}
  - Example of function call: fact(x);
- The formal parameters take the value of the current parameters
  - the current parameter can be an expression without an address
  - it can be of different compatible type → implicit conversion
- The information is (temporarily) duplicated

  → possible waste of CPU time and memory
- If I change the formal parameter, the actual/current parameter is not changed

  → passing of information only from the caller to the called

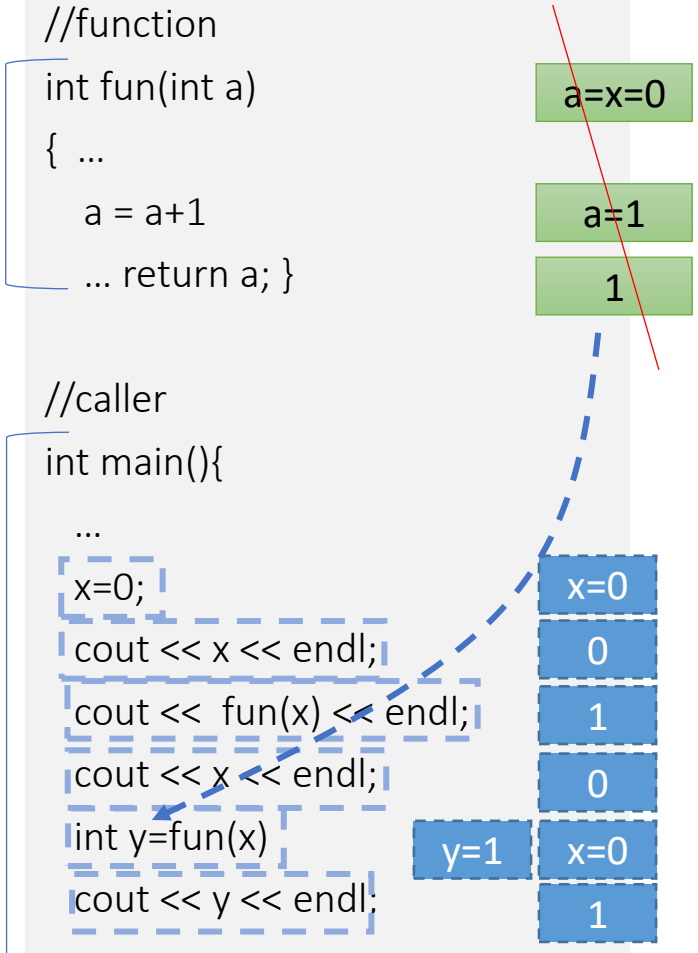
Examples:

{ ../FUNCTIONS/...}

Example

```
//function
int fun(int a)
{  …
   a = a+1
   … return a; }


//caller
int main(){
  …
  x=0;
  cout << x << endl;
  cout <<  fun(x) << endl;
  cout << x << endl;
  int y=fun(x)
  cout << y << endl;
  …
}
```



Parameter passing by reference

- The definition of formal parameters is analogous to definition of references
  - Syntax: type funct_name(type & identifier, ...)
  - E.g., int fun (int & n, int & m) {...}
  - Example of function call: fun(x,y);
- The parameter is the reference to the current parameter
  - the current parameter must be an expression with an address
  - the information is **not duplicated**
  - it must be of the same type

  → we avoid of loosing CPU time and memory
- If I change the formal parameter, the actual/current parameter is also changed

  → passing of information from the caller to the call

Parameter passing by reference

- The definition of formal parameters is analogous to definition of references
  - Syntax: type funct_name(type & identifier, ...)
  - E.g., int fun (int & n, int & m) {...}
  - Example of function call: fun(x,y);
- The parameter is the reference to the current parameter
  - the current parameter must be an expression with an address
  - the information is **not duplicated**
  - it must be of the same type

  → we avoid of loosing CPU time and memory
- If I change the formal parameter, the actual/current parameter is also changed

  → passing of information from the caller to the call

Parameter passing by reference

- The definition of formal parameters is analogous to definition of references
  - Syntax: type funct_name(type & identifier, ...)
  - E.g., int fun (int & n, int & m) {...}
  - Example of function call: fun(x,y);
- The parameter is the reference to the current parameter
  - the current parameter must be an expression with an address
  - the information is **not duplicated**
  - it must be of the same type

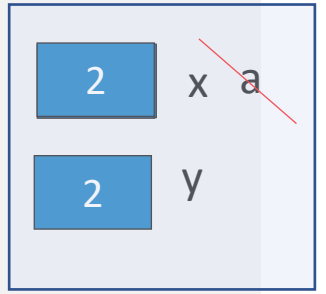
  → we avoid of loosing CPU time and memory
- If I change the formal parameter, the actual/current parameter is also changed

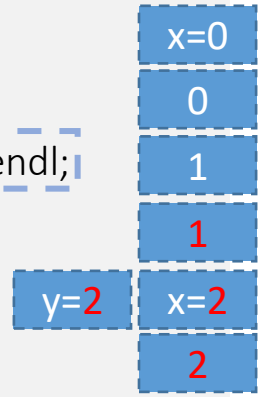
  → passing of information from the caller to the call

Example

```
//function
int fun(int& a)
{ ...
  a = a+1
  ... return a; }

//caller
int main(){
  ...
  x=0;
  cout << x << endl;
  cout <<  fun(x) << endl;
  cout << x << endl;
  int y=fun(x)
  cout << y << endl;
  ...
}
```





Parameter passing by reference

- The definition of formal parameters is analogous to definition of references
  - Syntax: type funct_name(type & identifier, ...)
  - E.g., int fun (int & n, int & m) {...}
  - Example of function call: fun(x,y);
- The parameter is the reference to the current parameter
  - the current parameter must be an expression with an address
  - the information is **not duplicated**
  - it must be of the same type

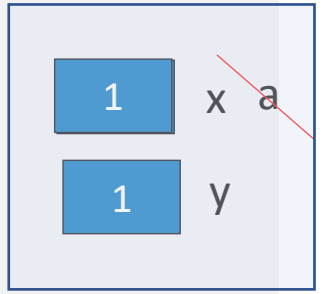
  → we avoid of loosing CPU time and memory
- If I change the formal parameter, the actual/current parameter is also changed

  → passing of information from the caller to the call

Example #2

```
//procedure
void fun(int& a)
{ ...
  a = a+1;
  ...}

//caller
int main(){
  ...
  x=0;
  cout << x << endl;
  fun(x);
  cout << x << endl;
  int y=x; ...
}
```



x=0

0

1

y=1

Examples (1)

Example of passing by value, wrong:

{ ../FUNCTIONS/**swap_err**.cc}

Example of passing by reference, correct [D]

{ ../FUNCTIONS/**swap**.cc}

Example of not admitted use of passing by reference, different type:

{ ../FUNCTIONS/reference_err.cc}

Example of problems in the use of passing by reference, when it is not expected :

{ ../FUNCTIONS/**gcf**.cc} { ../FUNCTIONS/**gcf_err**.cc}

Example of turning back two values:

{ ../FUNCTIONS/rectpolar.cc}

Example of parameter as input and output of a function:

{ ../FUNCTIONS/**vat**.cc}

Examples (1): vat.cc

```
1.  using namespace std;
2.  #include <iostream>


3.  void add_vat(float & val1, float  vat)       //val1 reference to val in main, i.e., val1=val=100, instead vat is a duplication of rate
4.  {
5.    val1 *= 1.0 + vat/100.0;   //it corresponds to: val1 = val1 * (1.0 + vat / 100.0) → 122
6.  }


7.  int main ()
8.  {
9.    float rate = 22.0f;
10.   float val;
11.   cout << "Value? ";
12.   cin >> val;    //100
13.  add_vat(val, rate);
14.   cout << "New value = " << val << endl;
15.   return 0;
16. }
```

```
$ g++ vat.cc
$ ./a.out
Value? 100
New value = 122
```

Examples (2): swap.cc

```
1.  using namespace std;
2.  #include <iostream>

3.  int a, b; // global variables
4.  void swap(int & n, int & m) {  //reference to n ref. to a=3, and m ref to b=2
5.   int temp;
6.   temp=n;              //temp=3
7.   n=m;                 //n=2
8.   m=temp;              //m=3
9.  }

10. int main(){
11.  do {
12.     cout << "insert a and b (0 0 for ends) ";
13.     cin >> a >> b ;   //3,2
14.     swap(a,b);
15.     cout << "a = " << a << " b= " << b << endl;          //->2,3
16.  } while ((a!=0)&&(b!=0));
17.  return 0;
18. }
```

```
$ g++ swap.cc
$ ./a.out
insert a and b (0 0 for ends) 3 2
a = 2 b= 3
insert a and b (0 0 for ends) 0 0
a = 0 b= 0
```

Examples (3): swap_err.cc

```
1.  using namespace std;
2.  #include <iostream>

3.  int a, b; // global variables
4.  void swap(int n, int m) {        //n copy of a=3, m copy of b=2
5.   int temp;
6.   temp=n;              //temp=3
7.   n=m;                 //n=2
8.   m=temp;              //m=3
9.   }

10. int main(){
11.  do {
12.     cout << "insert a and b (0 0 for ends) ";
13.     cin >> a >> b ;  //3,2
14.     swap(a,b);
15.     cout << "a = " << a << " b= " << b << endl;          //->3,2 no changes back form the function
16.  } while ((a!=0)&&(b!=0));
17.  return 0;
18. }
```

```
$ g++ swap_err.cc
$ ./a.out
insert a and b (0 0 for ends) 3 2
a = 3 b= 2
insert a and b (0 0 for ends) 0 0
a = 0 b= 0
```

Examples (4): swap_err.cc … fix

```
1.  using namespace std;
2.  #include <iostream>

3.  int a, b; // global variables
4.  void swap(int n, int m) {        //n copy of a=3, m copy of b=2
5.   int temp;
6.   temp=n;              //temp=3
7.   n=m;                 //n=2
8.   m=temp;               //m=3
9.   a =n;                //fix
10.  b=m ;                //fix
11. }

12. int main(){
13.  do {
14.     cout << "insert a and b (0 0 for ends) ";
15.     cin >> a >> b ;   //3,2
16.     swap(a,b);
17.     cout << "a = " << a << " b= " << b << endl;          //->3,2 no changes back form the function
18.   } while ((a!=0)&&(b!=0));
19.   return 0;
20. }
```

```
$ g++ swap_err.cc
$ ./a.out
insert a and b (0 0 for ends) 3 2
a = 2 b= 3
insert a and b (0 0 for ends) 0 0
a = 0 b= 0
```

Examples (5): gcf_err.cc (*greatest common factor*)

```
1.  ..
2.  int n1,n2;
3.  int gcf(int &  a, int & b){        //3,2 (a,b reference to n1 and n2)
4.     int rest;
5.     while(b!=0) {
6.        rest = a%b;                  //3%2=1    -> //2%1=1 -> //1%1=0
7.        a = b;                       //a=2      -> //a=1    -> //a=1
8.        b = rest;                    //b=1      ->  //b=1    -> //b=0 -> stop
9.     }
10.    return a;                                  //a=1
11. }

12. int main (){
13.    cout << "Insert a pair of numbers:  ";
14.    cin >> n1 >> n2;                                    //e.g., 3,2
15.    int res = gcf(n1,n2);                               //3,2 returned res=1
16.    cout << "The G.C.F. between " << n1 <<               //1
17.         " and " << n2 <<                               //0
18.        " is " << res << endl;                          // print 1
19.    return 0;
20. }
```

```
$ g++ gcf_err.cc
$ ./a.out
insert a pair of numbers: 3 2
The G.C.F. between 1 and 0 is 1
```

Examples (5): gcf_err.cc (*greatest common factor*)

```
1.  ..
2.  int n1,n2;
3.  int gcf(int &  a, int & b){        //3,2 (a,b reference to n1 and n2)
4.    int rest;
5.    while(b!=0) {
6.      rest = a%b;                    //3%2=1    -> //2%1=1 -> //1%1=0
7.      a = b;                         //a=2      -> //a=1    -> //a=1
8.      b = rest;                      //b=1      ->  //b=1    -> //b=0 -> stop
9.    }
10.   return a;                                   //a=1
11. }

12. int main (){
13.   cout << "Insert a pair of numbers:  ";
14.   cin >> n1 >> n2;                                    //e.g., 3,2
15.   int res = gcf(n1,n2);                               //3,2 returned res=1
16.   cout << "The G.C.F. between " << n1 <<               //1
17.        " and " << n2 <<                               //0
18.       " is " << res << endl;                          // print 1
19.   return 0;
20. }
```

```
$ g++ gcf_err.cc
$ ./a.out
insert a pair of numbers: 3 2
The G.C.F. between 1 and 0 is 1
```

Expected:
The G.C.F. between 3 and 2 is 1

Examples (5): gcf_fixed.cc (*greatest common factor*)

```
1.  ..
2.  int n1,n2;
3.  int gcf(int &  a,int & b){        //3,2 (a,b reference to n1 and n2)
4.    int rest;
5.    while(b!=0) {
6.      rest = a%b;                   //3%2=1   -> //2%1=1 -> //1%1=0
7.      a = b;                        //a=2     -> //a=1    -> //a=1
8.      b = rest;                     //b=1     ->  //b=1   -> //b=0 -> stop
9.    }
10.   return a;                                 //a=1
11. }

12. int main (){
13.   cout << "Insert a pair of numbers:  ";
14.   cin >> n1 >> n2;                                    //e.g., 3,2
15.   int res = gcf(n1,n2);                               //3,2 returned res=1
16.   cout << "The G.C.F. between " << n1 <<               //3
17.        " and " << n2 <<                               //2
18.       " is " << res << endl;                          // print 1
19.   return 0;
20. }
```

```
$ g++ gcf_err.cc
$ ./a.out
insert a pair of numbers: 3 2
The G.C.F. between 3 and 2 is 1
```

Parameter passing by (constant) reference

- It is possible to define passing by reference in reading only (by constant reference)
  - Syntax: type funct_name(const type & identifier, ...)
  - E.g., int fact (const int & n, …) {...}
  - Example of function call: fact(x, …);
- It does not allow for modification of the n parameter
  - E.g., n =5; //wrong
    → passing of information only to the calling to the callee
    → only in input to the function
- Reference: the information is not duplicated

  → we avoid to lose CPU time and memory
- It is used to pass (input) to a function “big object”
  - It avoids errors
  - It is efficient (no loose of CPU time and memory)
  - It allows to easy identify the function inputs

Examples:

{ ../FUNCTIONS/use_const.cc}

Parameter passing by (constant) reference

- It is possible to define passing by reference in reading only (by constant reference)
  - Syntax: type funct_name(const type & identifier, ...)
  - E.g., int fact (const int & n, …) {...}
  - Example of function call: fact(x, …);
- It does not allow for modification of the n parameter
  - E.g., n =5; //wrong
    → passing of information only to the calling to the callee
    → only in input to the function
- Reference: the information is not duplicated
  → we avoid to lose CPU time and memory
- It is used to pass (input) to a function “big object”
  - It avoids errors
  - It is efficient (no loose of CPU time and memory)
  - It allows to easy identify the function inputs

Examples:
{ ../FUNCTIONS/use_const.cc}

Example

```
//function
void fun(int& a, const int &b)
{  …
   a= 10;
   b= 20;
}

//caller
int main(){
  …
  int x=20; int b=40;
  fun(x,b);
  cout << a << endl;
  cout << b << endl;
}
```

//compile-time error!

Examples (2)

Example of passing by reference:

{ ../FUNCTIONS/cipciop.cc}

{ ../FUNCTIONS/goofy.cc}

{ ../FUNCTIONS/donaldduck.cc}

{ ../FUNCTIONS/mickeymouse.cc}

Example – Nested function call and reference to

```
1.  using namespace std;
2.  #include <iostream>

3.  int a,b;

4.  void fun2(int & t,int & s)
5.      //t ref to y (y ref to b) (4), s ref to x (x ref to a) (5)
6.  {
7.    t = s;   //t = s (5)
8.    s += 2;  //s = s (5) +2 =7
9.  }

10. void fun1(int & x,int & y) //x ref to a (5), y ref to b (4)
11. {
12.   fun2(y,x);  // y ref to b (4), x ref to a (5)
13. }
```

```
14. int main ()
15. {
16.   a=5;
17.   b=4;
18.   cout << "a= " << a << ", b= " << b << endl; //a=5,b=4
19.   fun1(a,b);
20.   cout << "a= " << a << ", b= " << b << endl; //a=7,b=5
21.   return 0;
22. }
```

```
$ g++ goofy.cc
$ ./a.out
...
```

Example – Nested function call and reference to

```
1.  using namespace std;
2.  #include <iostream>

3.  int a,b;

4.  void fun2(int & t,int & s)
5.      //t ref to y (y ref to b) (4), s ref to x (x ref to a) (5)
6.  {
7.    t = s;   //t = s (5)
8.    s += 2;  //s = s (5) +2 =7
9.  }

10. void fun1(int & x,int & y) //x ref to a (5), y ref to b (4)
11. {
12.   fun2(y,x);  // y ref to b (4), x ref to a (5)
13. }
```

```
14. int main ()
15. {
16.   a=5;
17.   b=4;
18.   cout << "a= " << a << ", b= " << b << endl; //a=5,b=4
19.   fun1(a,b);
20.   cout << "a= " << a << ", b= " << b << endl; //a=7,b=5
21.   return 0;
22. }
```

```
$ g++ goofy.cc
$ ./a.out
a= 5, b= 4
```

Example – Nested function call and reference to

```
1.  using namespace std;
2.  #include <iostream>

3.  int a,b;

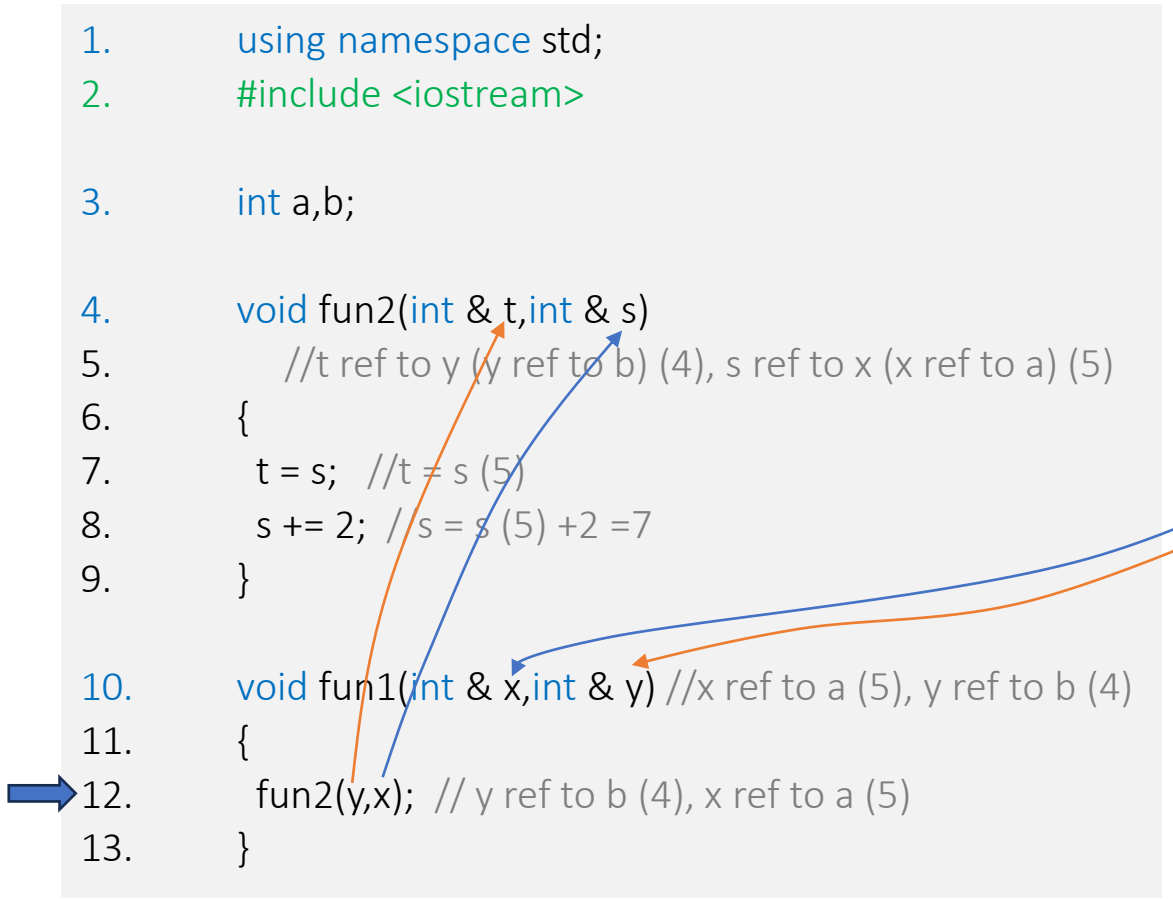
4.  void fun2(int & t,int & s)
5.      //t ref to y (y ref to b) (4), s ref to x (x ref to a) (5)
6.  {
7.    t = s;   //t = s (5)
8.    s += 2;  //s = s (5) +2 =7
9.  }

10. void fun1(int & x,int & y) //x ref to a (5), y ref to b (4)
11. {
12.   fun2(y,x);  // y ref to b (4), x ref to a (5)
13. }
```

```
14. int main ()
15. {
16.   a=5;
17.   b=4;
18.   cout << "a= " << a << ", b= " << b << endl; //a=5,b=4
19.   fun1(a,b);
20.   cout << "a= " << a << ", b= " << b << endl; //a=7,b=5
21.   return 0;
22. }
```

```
$ g++ goofy.cc
$ ./a.out
a= 5, b= 4
```

Example – Nested function call and reference to



```
1.  using namespace std;
2.  #include <iostream>

3.  int a,b;

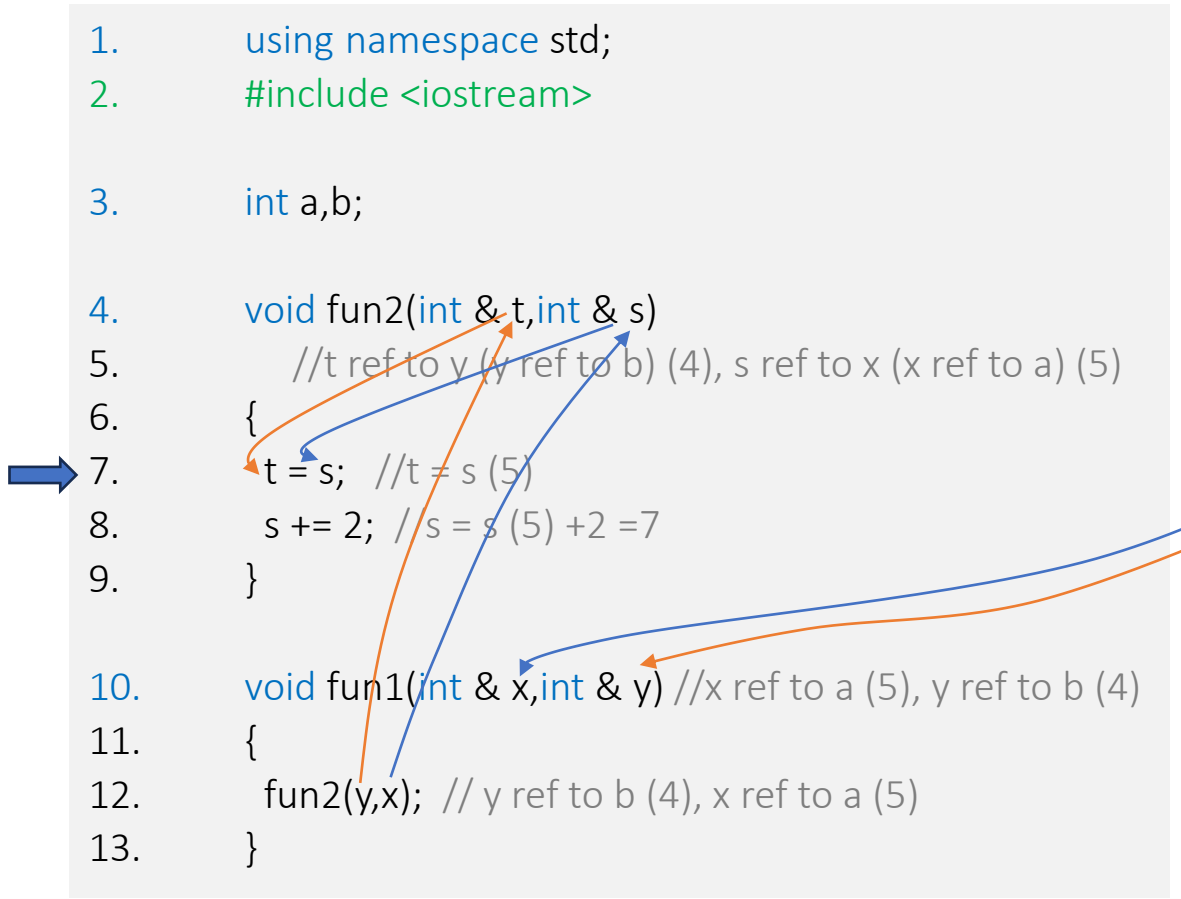
4.  void fun2(int & t,int & s)
5.      //t ref to y (y ref to b) (4), s ref to x (x ref to a) (5)
6.  {
7.    t = s;   //t = s (5)
8.    s += 2;  // s = s (5) +2 =7
9.  }

10. void fun1(int & x,int & y) //x ref to a (5), y ref to b (4)
11. {
12.   fun2(y,x);  // y ref to b (4), x ref to a (5)
13. }
```

```
14. int main ()
15. {
16.   a=5;
17.   b=4;
18.   cout << "a= " << a << ", b= " << b << endl; //a=5,b=4
19.   fun1(a,b);
20.   cout << "a= " << a << ", b= " << b << endl; //a=7,b=5
21.   return 0;
22. }
```

```
$ g++ goofy.cc
$ ./a.out
a= 5, b= 4
```

Example – Nested function call and reference to



```
1.  using namespace std;
2.  #include <iostream>

3.  int a,b;

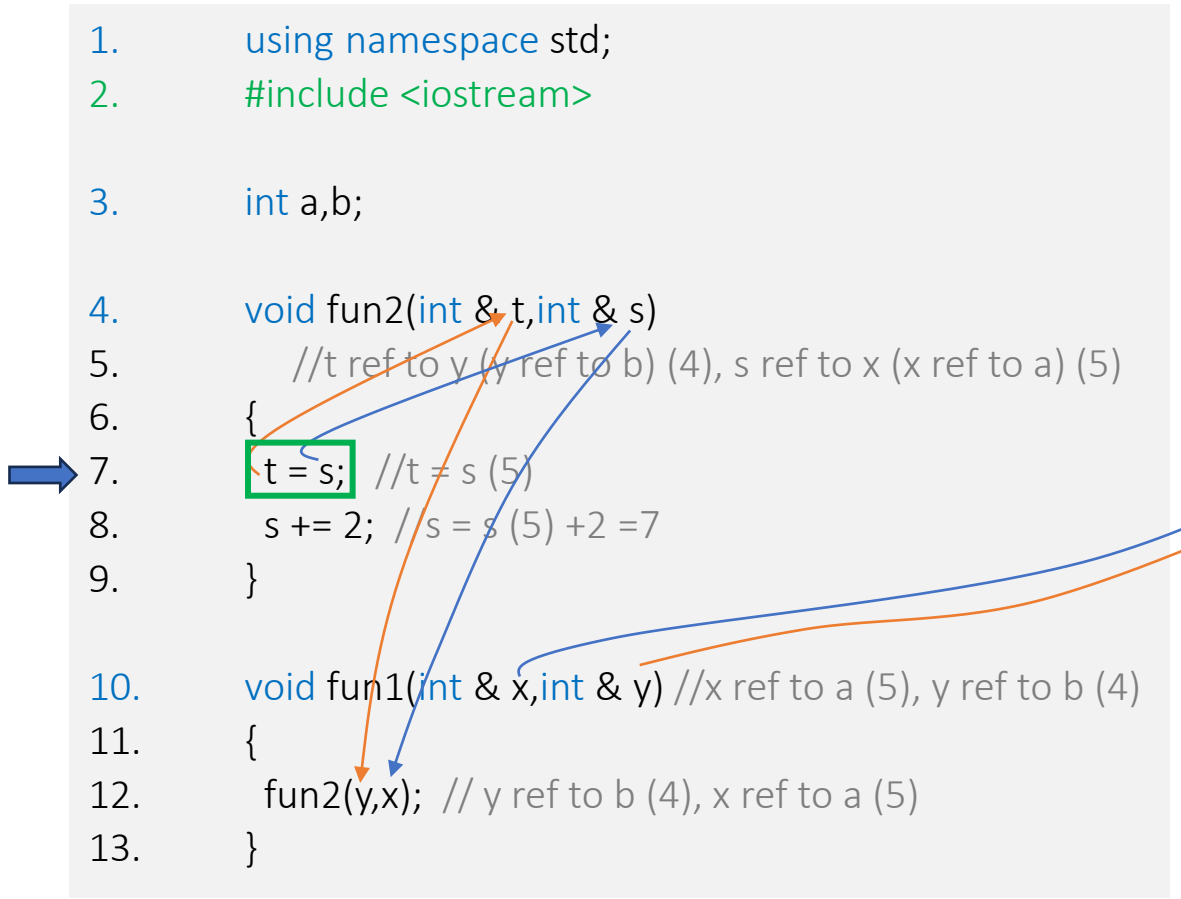
4.  void fun2(int & t,int & s)
5.      //t ref to y (y ref to b) (4), s ref to x (x ref to a) (5)
6.  {
7.   t = s;   //t = s (5)
8.    s += 2;  // s = s (5) +2 =7
9.  }

10. void fun1(int & x,int & y) //x ref to a (5), y ref to b (4)
11. {
12.  fun2(y,x);  // y ref to b (4), x ref to a (5)
13. }
```

```
14. int main ()
15. {
16.   a=5;
17.   b=4;
18.   cout << "a= " << a << ", b= " << b << endl; //a=5,b=4
19.   fun1(a,b);
20.   cout << "a= " << a << ", b= " << b << endl; //a=7,b=5
21.   return 0;
22. }
```

```
$ g++ goofy.cc
$ ./a.out
a= 5, b= 4
```

Example – Nested function call and reference to



```
1.  using namespace std;
2.  #include <iostream>

3.  int a,b;

4.  void fun2(int & t,int & s)
5.      //t ref to y (y ref to b) (4), s ref to x (x ref to a) (5)
6.  {
7.   t = s;   //t = s (5)
8.    s += 2;  // s = s (5) +2 =7
9.  }

10. void fun1(int & x,int & y) //x ref to a (5), y ref to b (4)
11. {
12.   fun2(y,x);  // y ref to b (4), x ref to a (5)
13. }
```

```
14. int main ()
15. {
16.   a=5;
17.   b=4;
18.   cout << "a= " << a << ", b= " << b << endl; //a=5,b=4
19.   fun1(a,b);
20.   cout << "a= " << a << ", b= " << b << endl; //a=7,b=5
21.   return 0;
22. }
```

```
$ g++ goofy.cc
$ ./a.out
a= 5, b= 4
```

Example – Nested function call and reference to

```
1.  using namespace std;
2.  #include <iostream>

3.  int a,b;

4.  void fun2(int & t,int & s)
5.      //t ref to y (y ref to b) (4), s ref to x (x ref to a) (5)
6.  {
7.    t = s;   //t = s (5)
8.    s += 2;  //s = s (5) +2 =7
9.  }

10. void fun1(int & x,int & y) //x ref to a (5), y ref to b (4)
11. {
12.   fun2(y,x);  // y ref to b (4), x ref to a (5)
13. }
```

```
14. int main ()
15. {
16.   a=5;
17.   b=4;
18.   cout << "a= " << a << ", b= " << b << endl; //a=5,b=4
19.   fun1(a,b);
20.   cout << "a= " << a << ", b= " << b << endl; //a=7,b=5
21.   return 0;
22. }
```

```
$ g++ goofy.cc
$ ./a.out
a= 5, b= 4
```

Example – Nested function call and reference to

```
1.  using namespace std;
2.  #include <iostream>

3.  int a,b;

4.  void fun2(int & t,int & s)
5.      //t ref to y (y ref to b) (4), s ref to x (x ref to a) (5)
6.  {
7.    t = s;   //t = s (5)
8.    s += 2;  //s = s (5) +2 =7
9.  }

10. void fun1(int & x,int & y) //x ref to a (5), y ref to b (4)
11. {
12.   fun2(y,x);  // y ref to b (4), x ref to a (5)
13. }
```

```
14. int main ()
15. {
16.   a=5;
17.   b=4;
18.   cout << "a= " << a << ", b= " << b << endl; //a=5,b=4
19.   fun1(a,b);
20.   cout << "a= " << a << ", b= " << b << endl; //a=7,b=5
21.   return 0;
22. }
```

```
$ g++ goofy.cc
$ ./a.out
a= 5, b= 4
```

Example – Nested function call and reference to

```
1.  using namespace std;
2.  #include <iostream>

3.  int a,b;

4.  void fun2(int & t,int & s)
5.      //t ref to y (y ref to b) (4), s ref to x (x ref to a) (5)
6.  {
7.    t = s;   //t = s (5)
8.    s += 2;  //s = s (5) +2 =7
9.  }

10. void fun1(int & x,int & y) //x ref to a (5), y ref to b (4)
11. {
12.   fun2(y,x);  // y ref to b (4), x ref to a (5)
13. }
```

```
14. int main ()
15. {
16.   a=5;
17.   b=4;
18.   cout << "a= " << a << ", b= " << b << endl; //a=5,b=4
19.   fun1(a,b);
20.   cout << "a= " << a << ", b= " << b << endl; //a=7,b=5
21.   return 0;
22. }
```

```
$ g++ goofy.cc
$ ./a.out
a= 5, b= 4
a= 7, b= 5
```

Parameter passing by pointer/address

- The definition of formal parameters: pointers are passed by value
  - Syntax: type funct_name(type * identifier, ...)
  - E.g., int swap (int * pn, int * pm) {...}
  - In the function call, the address of the object is passed
  - Example of function call: swap(&x);
- The parameter is a pointer to the (object which address is given by the) current parameter
  - It must be an expression with an address
  - It must be of the same type
  - The information is **not duplicated**

  → we avoid of loosing CPU time and memory
- If I change the formal parameter, the current parameter is also changed

  → passing of information also from the called to the caller

(the effect is similar to the one of the passing by reference)

Parameter passing by pointer/address

- The definition of formal parameters: pointers are passed by value
  - Syntax: type funct_name(type * identifier, ...)
  - E.g., int swap (int * pn, int * pm) {...}
  - In the function call, the address of the object is passed
  - Example of function call: swap(&x);
- The parameter is a pointer to the (object which address is given by the) current parameter
  - It must be an expression with an address
  - It must be of the same type
  - The information is **not duplicated**

  → we avoid of loosing CPU time and memory
- If I change the formal parameter, the current parameter is also changed

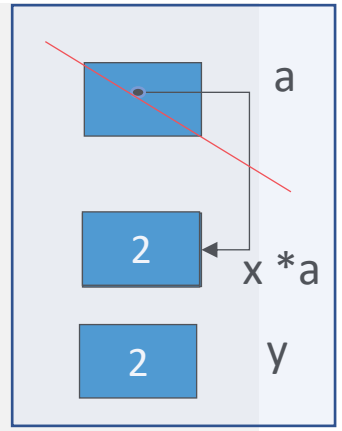
  → passing of information also from the called to the caller

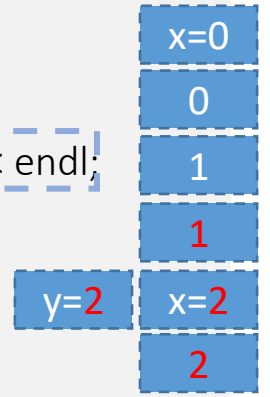
(the effect is similar to the one of the passing by reference)

Example

```
//function
int fun(int *a)
{ ...
  *a = *a+1
  ... return *a; }

//caller
int main(){
  ...
  x=0;
  cout << x << endl;
  cout <<  fun(&x) << endl;
  cout << x << endl;
  int y=fun(&x)
  cout << y << endl;
  ...
}
```





Examples (3)

Example of passing by pointer:

{ ../FUNCTIONS/swap_point.cc}

{ ../FUNCTIONS/vat2.cc}

{ ../FUNCTIONS/donaldduck2.cc}

Example (3): vat2.cc (1)

```
using namespace std;
#include <iostream>

void add_vat(float * pval, float  vat)  //pval points to the address of val, vat copy of rate
{
  *pval *= 1.0 + vat/100.0;  //different meaning of the * → *pval = *pval * (1.0+ vat/100) → 122
}

int main ()
{
  float rate = 22;
  float val;
  cout << "Value? ";
  cin >> val;   //100
  add_vat(&val, rate);
  cout << "New value = " << val << endl;
  return 0;
}
```

```
$ g++ vat2.cc
$ ./a.out
Value? 100
New value = 122
```

Example (3): vat2.cc (differences)

by pointer/address

```
using namespace std;
#include <iostream>

void add_vat(float * pval, float  vat)
{
  *pval *= 1.0 + vat/100.0;
}

int main ()
{
  float rate = 22;
  float val;
  cout << "Value? ";
  cin >> val;
  add_vat(&val, rate);
  cout << "New value = " << val << endl;
  return 0;
}
```

Example (3): vat2.cc (differences)

by pointer/address

```
using namespace std;
#include <iostream>

void add_vat(float * pval, float  vat)
{
  *pval *= 1.0 + vat/100.0;
}

int main ()
{
  float rate = 22;
  float val;
  cout << "Value? ";
  cin >> val;
  add_vat(&val, rate);
  cout << "New value = " << val << endl;
  return 0;
}
```

by reference

```
using namespace std;
#include <iostream>

void add_vat(float & val, float  vat)
{
  val *= 1.0 + vat/100.0;
}

int main ()
{
  float rate = 22;
  float val;
  cout << "Value? ";
  cin >> val;
  add_vat(val, rate);
  cout << "New value = " << val << endl;
  return 0;
}
```

Parameter passing by values vs by reference/pointer

Advantage of passing by reference / pointer:

- Less CPU computation and memory load (especially with large parameters)
- It allows to return information from the call to the caller

Disadvantages of passing by reference / pointer:

- Risk of code confusion (you don't know where the values change)
- Aliasing (entity with more than one name)
- Formal and actual parameter of exactly the same type
- Only expressions with the address can be passed

Functions that return a reference (1)

- It returns a reference to an expression with address
  - the expression must refer to an object of the caller (e.g., a formal parameter passed by reference, an element of an array)
  - it must be of the same type
- The call is an expression with an address!

Example

```
int number=5;
int& setNumber() {     // it returns the address of the number variable
        //return 10; //wrong! Compile-time error! Cannot be done since it returns an address
        return number;
}
int main() {
  int returnedNumber=setNumber(); //call to the function → returnedNumber=5
  cout << setNumber();  //5
  setNumber() = 10;  // 10 is assigned to the returned variable addess, i.e., number=10
  cout << number;  //10
  return 0;
```

Example: { ../FUNCTIONS/return_reference0.cc, return_reference.cc}

Functions that return a reference (2)

Example (2)

```
int& max(int& x, int& y)          //parameters are refereces and it also returns a reference
{
    return (x > y ? x : y);        //x,y are references to non-local objects
                                   //i.e., it returns the reference to x or y, depending on the condition
}

...
int m=44, n=22;
max(m,n) = 55;          //it changes the value of m from 44 to 55 (m=55)
//i.e., it assigns to the address variable returned by the function max, executed with m and n, the value 55

m=22, n=44;
max(m,n) = 55;          //it changes the value of n da from 44 to 55 (n=55)
// i.e., it assigns to the address variabale returned by the function max, executed with m and n, the value 55
```

Example

{ ../FUNCTIONS/return_reference0.cc, return_reference.cc}

Functions that return a reference (2) - example

```
using namespace std;
#include <iostream>

int& max(int& x, int& y)  // return type is reference to int
{
  return (x > y ? x : y); // m and n are non-local references
}

int main()
{
  int m = 44, n = 22;
  cout << "(0) " << m << ", " << n << ", " << max(m,n) << endl;
  max(m,n) = 55;       // changes the value of m from 44 to 55
  cout << "(1) “ << m << ", " << n << ", " << max(m,n) << endl;

  m = 22, n = 44;
  cout << "(2) " << m << ", " << n << ", " << max(m,n) << endl;
  max(m,n) = 55;       // changes the value of n from 44 to 55
  cout << "(3) " << m << ", " << n << ", " << max(m,n) << endl;
  return 0;
}
```

```
$ g++ return_reference0.cc
$ ./a.out
(0) 44, 22, 44
(1) 55, 22, 55
(2) 22, 44, 44
(3) 22, 55, 55
```

{ ../FUNCTIONS/return_reference0.cc}

Overloading of parameters

- In C ++ it is possible to give the same name to different functions, but if they have lists of different parameters (different by number and/or by type)
  - The compiler "recognizes" the right function for each call
  - If there is any ambiguity, the compiler produces an error
  - Implicit conversions are allowed, if they do not cause ambiguity

Example

```
int max(int, int) {...};
int max(int, int, int) {...};
double max(double, double) {...};
...
cout << max(99,77) << " " << max(55,66,33) << " “ << max(3.4,7.2) << endl;     // ok
cout << max(3,3.1) << endl;                                                    // wrong: ambiguity
```

Example of above:

{ ../FUNCTIONS/overloading.cc}

It is not possible to overload functions distinguished only by return type

Functions with default arguments

- In C ++ it is possible to provide optional parameters, with default values
  - It allows calls with reduced list of parameters
  - The optional parameters must be the last of the parameter's list
  - The match is done from left to right

Example 1

```
double p2(double x=1.1)
{
    return x;
}

double x=2.0003;
cout << p2(x) << endl;  //2.0003
cout << p2() << endl; //1.1  //note: this is not possible
                              // without the default value for x
```

Example 2

```
double p(double, double, double =0, double =0, double =0);

double p(double x, double a0, double a1, double a2, double a3)
{
    return a0 + (a1 + (a2 + a3*x)*x)*x;
}
double x=2.0003;
cout << p(x) << endl; //wrong
cout << p(x, 7) << endl; //7
cout << p(x, 7, 6) << endl; //19.0018
cout << p(x, 7, 6, 5) << endl; //39.0078
cout << p(x, 7, 6, 5, 4) << endl; //71.0222
```

Example: { ../FUNCTIONS/defaultvalues.cc} { ../FUNCTIONS/defaultvalues2.cc}